



TechCouncil
of Australia

Agentic Engineering

Codespaces Setup

Codespaces Setup — TCA Agentic Engineering

Read this instead of *Before the day* in the Student Handout if you cannot install software on your laptop. Everything else in the handout still applies unchanged: the prompts, the command map, the golden rules. This document covers the setup, the four places Codespaces behaves differently, and the handful of things it will not do at all.

A **codespace** is a Linux container GitHub runs for you, edited through VS Code in a browser tab. Nothing lands on your laptop, so no admin rights are needed. You get a real terminal, so both agents run normally. The trade is that the container is somewhere else, which is what makes browser logins, forwarded ports and long pastes behave oddly.

Windows and Mac are the same document. The work happens in a browser tab against a Linux container, so the only differences are which modifier key you press. Where it matters, both are given.

What you need, and what it costs

A personal GitHub account on the Free plan is enough for the Codespaces side. **A paid Claude plan is not optional:** Claude Code needs Pro, Max, Team, Enterprise or Console, and the free Claude.ai plan does not include it. Sort that out before the day or you will get through Step 2 and stall at Step 3.

Chrome or Edge is the path of least resistance, and the one GitHub recommends. Firefox and Safari do work, and hit more known issues; in Firefox the command palette shortcut is taken by the browser.

Plan	Compute	Storage	Real hours on a 2-core machine
GitHub Free	120 core-hours / month	15 GB-month	About 60, because a 2-core machine burns 2 core-hours per hour
GitHub Pro	180 core-hours / month	20 GB-month	About 90 on 2-core, about 45 on 4-core

Core-hours, not hours. Active time is multiplied by the core count. The course day is roughly seven hours of running container, so about 14 core-hours on a 2-core machine. That is comfortable inside the free allowance even if you spend a second day practising. Pick 4 cores at creation if you want the two-agents-at-once section to feel smooth, and accept double the burn.

- **Read the core count off the create dialog** rather than assuming. GitHub gives you the smallest machine that is valid, and a repository or an org policy can raise that floor.
- Storage accrues for as long as the codespace **exists**, running or stopped. Delete it when you are finished with it, not just stop it.
- Run out of allowance with no payment method on file and GitHub blocks new and resumed codespaces rather than charging you. You can still export your work to a branch. With a card on file it bills instead, so set a budget if that worries you.

- **Closing the browser tab does not stop the codespace.** It keeps running and keeps billing until the idle timeout, 30 minutes by default.

Step 1 · Create one codespace, on the reference repo

Create it on the reference repo, not on an empty one. GitHub cannot create a codespace for a repository with no files, and starting from the reference gets the safety net cloned for free. Your own app goes in a second directory inside the same container, which is cheaper and faster than juggling two codespaces in two tabs.

1. Sign in at **github.com**.
2. Go to `github.com/paulkeen/tca-habits`.
3. Click the green **Code** button, then the **Codespaces** tab, then **Create codespace on main**.
4. To choose 4 cores instead of 2: in that same Codespaces tab use the ... menu, **New with options**, then set **Machine type**. Do it now; changing it later needs a stop and restart.
5. Wait for the container to build. A browser VS Code opens with a terminal at the bottom. If there is no terminal, open one from the **Terminal › New Terminal** menu.

The repo is cloned to `/workspaces/tca-habits`. That is the safety net. The handout calls it `~/tca-habits-reference`; translate any handout path that starts with `~/` to `/workspaces/`.

TERMINAL · MAKE THE FOLDER FOR YOUR OWN APP

```
mkdir -p /workspaces/my-habits
cd /workspaces/my-habits && git init

# /workspaces is the only directory that survives both a stop and a rebuild.
# Do not build your app in ~/ or /tmp.
```

To see both folders in the Explorer, use **File › Add Folder to Workspace...** and pick `/workspaces/my-habits`. The agent itself only cares about the terminal's working directory, so this is cosmetic.

	Where	What it is
Your app	<code>/workspaces/my-habits</code>	Empty at 10:00. You build the whole thing here, by prompt.
The net	<code>/workspaces/tca-habits</code>	Already cloned when the codespace was created. You only open this if you fall behind.

Step 2 · Install the toolchain

The default container image already has Node, Python, git, npm and the GitHub CLI. It does not have uv. Expect Node 24 and Python 3.14, both newer than the course needs, and check the versions before you install anything. No sudo anywhere in this step.

TERMINAL

```
# Everything below must print a version, not an error.
node --version          # expect 24.x
python3 --version       # expect 3.14.x
git --version

# uv is not preinstalled. No admin rights needed.
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv --version

# One agent, or both. Either is fine; the ideas are identical.
npm install -g @anthropic-ai/claude-code && claude --version
npm install -g @openai/codex && codex --version
```

If npm complains about the Node engine, use Anthropic's native installer instead: `curl -fsSL https://claude.ai/install.sh | bash`. It is now the recommended method and does not care what Node you have.

This step is the one that evaporates. Everything outside `/workspaces` is tied to the container, so it survives **stop and start** but not **Rebuild Container**. Avoid the rebuild command. If you do rebuild, come back and repeat Step 2 and Step 3, and re-run `/terminal-setup`.

Step 3 · Log in to the agents

This is the step that catches people, and the reason is the same for both tools. The agent starts a small web server on the container's `localhost` and expects the browser to redirect back to it after you sign in. Your browser is on your laptop, so the redirect has nowhere to land. Both vendors ship a documented way around it.

Claude Code

1. Run `claude`. Choose your login option.
2. If no browser opens, press **c** to copy the authorisation URL, then paste it into a browser tab yourself.
3. Sign in. The browser shows a **login code** instead of redirecting back. That is the expected behaviour in a container.
4. Paste that code at the `Paste code here` if prompted prompt in the terminal.
5. If pasting into the interactive prompt does nothing, quit and run `claude auth login`, which prints the URL and reads the code from standard input.

OpenAI Codex

1. Run `codex login` and choose **Sign in with Device Code**, or go straight there with `codex login --device-auth`.
2. Device code login can be disabled for your account or workspace. If it is, Codex falls back to browser login, which will not complete in a codespace.
3. Fallback: sign in on a machine that has a browser and the CLI, then copy its `~/.codex/auth.json` into the codespace. Treat that file like a password; it holds access tokens.

4. Fallback: `printenv OPENAI_API_KEY | codex login --with-api-key`. Note that API-key sign-in bills your OpenAI Platform account at API rates, not your ChatGPT plan, and disables the features that draw on ChatGPT credits.

Make the login survive, if you want to

`~/.claude` persists across stop and start but is cleared on rebuild, so a rebuild logs you out. To make the login automatic, store a credential as a Codespaces secret at **Settings > Codespaces > Secrets**; Codespaces exposes secrets as environment variables inside the container. Either `ANTHROPIC_API_KEY`, or a year-long token from `claude setup-token` stored as `CLAUDE_CODE_OAUTH_TOKEN`.

The cost of doing that: when an API key or a token is the active credential, `claude.ai` connectors are unavailable. If the calendar-MCP stretch exercise matters to you, sign in interactively instead.

Step 4 · Warm the safety net

TERMINAL

```
cd /workspaces/tca-habits
./scripts/setup.sh          # backend venv (uv) + frontend deps (npm)
git tag                    # checkpoint-1-working / -2-features / -3-hardened
```

The checkpoint reset commands in the handout work unchanged. Only the directory differs: `cd /workspaces/tca-habits` in place of `cd ~/tca-habits-reference`.

Step 5 · Run the app and actually see it

The second real difference. The dev servers listen on the container's `localhost`, and `localhost:5173` in your own browser is your laptop, which is running nothing. Codespaces forwards the port and gives you a URL instead.

1. Start the app: `./scripts/dev.sh`, or make `dev` from `checkpoint-2-features` onwards.
2. The script prints `http://localhost:8000` and `http://localhost:5173`. Codespaces watches for exactly that and forwards both ports automatically.
3. Open the **PORTS** tab next to the terminal. Find port 5173 and click the globe icon, or **Open in Browser**.
4. The URL looks like `https://<codespace-name>-5173.app.github.dev`. Never type `localhost` into your own browser.
5. You only ever need port 5173. The Vite proxy forwards `/habits` and `/stats` to port 8000 from inside the container, so CORS stays invisible.

Leave the ports private. Private is the default and means only you can open them. Public means anyone with the URL, and there is no auth in front of this app.

If the page says Blocked request. This host is not allowed, that is Vite's host check, not Codespaces. Vite validates the `Host` header against `server.allowedHosts` and the forwarded

domain is not in the default list. Fix it in `frontend/vite.config.ts`, or better, describe the symptom to the agent and let it fix it.

FRONTEND/VITE.CONFIG.TS

```
server: {
  port: 5173,
  allowedHosts: ['.app.github.dev'], // not `true`: that is a rebinding hazard
  proxy: { '/habits': 'http://localhost:8000', '/stats': 'http://localhost:8000' },
},
```

What Codespaces will not do

Four of these matter on the day. The rest are listed so you do not waste a lab block finding out the hard way.

The four that will bite you

What	Why	What to do instead
Pasting the long prompts	Anthropic's own docs: the VS Code integrated terminal can drop characters from very large pastes before they reach Claude Code. The browser terminal is that terminal.	Write the prompt to a file and tell the agent to read it: <code>nano prompt.txt</code> , then <code>Read prompt.txt</code> and do what it says. For anything over about twenty lines, do this by default.
Shift+Enter for a newline	The VS Code terminal does not send it in a form Claude Code recognises until you configure it.	<code>Ctrl+J</code> inserts a newline with no setup at all, in any terminal. For Shift+Enter proper, run <code>/terminal-setup</code> once inside Claude Code; it also turns off GPU acceleration to stop the garbled-text problem, and a rebuild undoes it.
Browser-reserved keystrokes	The browser gets the keypress before VS Code does.	See the table below. If plan mode or the command palette will not respond, install the codespace as a progressive web app, which is the documented workaround.
Codex's own sandbox	Codex isolates with <code>bwrap</code> plus <code>seccomp</code> . Inside a container those namespace operations may be blocked, and Codex will warn or fail at startup.	OpenAI's documented answer: the container is already your boundary, so run <code>codex --sandbox danger-full-access</code> and do not try to nest a second sandbox.

You press	The browser takes it	Use instead
<code>Ctrl+Shift+P</code> , Mac <code>Shift+Cmd+P</code>	Reserved in Firefox	<code>F1</code>
<code>Ctrl+W</code> , Mac <code>Cmd+W</code>	Closes the browser tab	<code>Ctrl+Shift+Alt+N</code>
<code>Ctrl+N</code> , Mac <code>Cmd+N</code>	Opens a browser window	<code>Ctrl+Alt+N</code>

You press	The browser takes it	Use instead
Alt+Left / Alt+Right	May trigger browser tab history instead of editor navigation, but only when focus is outside the editor	Click into the editor first, or use Go Back from the palette
F10 / F11 when debugging	Step over is remapped in every browser. Step into is remapped on Windows. On macOS F11 is taken by the system.	Alt+F10 to step over. Alt+F11 to step into on Windows. On macOS, turn off System Settings › Keyboard › Shortcuts › Show Desktop F11 .

Gone entirely, and not needed on the day

- **Claude Code's /chrome browser integration.** It needs a Chrome or Edge install on the same machine plus a native messaging host file in a local browser profile. A container has neither.
- **Computer use in the CLI.** Documented as macOS only, and explicitly unavailable on Linux.
- **Voice dictation, image paste, /copy, desktop notifications.** All of these reach for a host clipboard, microphone or notification channel. Set preferredNotifChannel to terminal_bell if you want to hear a turn finish.
- **VS Code Desktop as the editor** unless you can install it. See the optional section below.

Works, but needs an extra step

Thing	The extra step
Remote MCP servers with OAuth	Same localhost-callback problem as login, and the same shape of fix. Run <code>claude mcp login <name> --no-browser</code> to print the URL, then paste the full callback URL from your browser's address bar back into Claude Code. If the server needs a pre-registered redirect URI, pin the port when you register it: <code>claude mcp add ... --callback-port</code> . Anthropic-hosted connectors such as Google Calendar cannot be connected from the CLI at all; do it at claude.ai first, and note that this only works if you signed in interactively rather than with a key.
Local stdio MCP servers	Whatever binary the server shells out to has to exist inside the container. Install it in the terminal, and put the server definition in a checked-in <code>.mcp.json</code> at the repo root so it comes back after a rebuild.
Git worktrees, for the two-agents-at-once section	The mechanics are unchanged. Two gotchas. Claude Code's <code>--worktree</code> flag needs the workspace already trusted, so run <code>claude</code> once in the directory and accept the trust prompt first. And in Codex's default workspace-write sandbox the resolved <code>.git</code> directory of a worktree is read-only, so commits from inside a worktree will ask for approval.
Pushing your app to GitHub, for the hooks and CI block	<code>/workspaces/my-habits</code> has no remote until you make one. Do not assume <code>gh</code> can create it: the token a codespace hands you is normally scoped to the repository it was created from, so <code>gh repo create</code> may be refused. Run <code>gh auth login</code> first, or create the empty repo in the GitHub web UI and then <code>git remote add origin ...</code> and <code>git push -u origin main</code> .

Thing	The extra step
Running the agent without permission prompts	The container is a reasonable boundary, and both vendors bless it with the same caveat. Claude Code: <code>--dangerously-skip-permissions</code> , restricted by the docs to isolated containers and VMs without internet access , which a codespace is not. Codex: <code>--sandbox danger-full-access</code> . Read the caveat literally. The container stops the agent damaging your laptop; it does not stop a malicious repository reading the agent credentials sitting in <code>~/ .claude</code> or <code>~/ .codex</code> and posting them somewhere.
The AI features in Section 2	The weekly summary and encouragement code paths call Claude when <code>ANTHROPIC_API_KEY</code> is set, and fall back to templates when it is not. Set it as a Codespaces secret rather than pasting it into a terminal, where it lands in your shell history.

Housekeeping, and the two ways to lose your work

Codespaces deletes things on a timer. Neither default is unreasonable, and both will catch you if you have not read them once.

Timer	What it does
Idle timeout	Stops the codespace after 30 minutes of no keyboard, mouse or terminal output. Configurable from 5 to 240 minutes at Settings › Codespaces › Default idle timeout , and only for new codespaces. A stop is harmless: uncommitted work survives, and you restart by clicking the codespace name.
Retention	Deletes a stopped codespace after 30 days , which is also the maximum. It deletes even if you have unpushed commits . Use Keep codespace from the kebab menu at github.com/codespaces to hold one indefinitely, and remember storage keeps accruing.

Operation	What you keep
Stop and start	/workspaces, the CLIs you installed, your logins, your shell history. Everything except /tmp, which is cleared on every stop.
Rebuild Container	Only /workspaces and /tmp. The CLIs, uv, your logins, /terminal-setup and any shell config are gone. Avoid this command .
Delete	Nothing. Push first.

At the end of the day: stop the codespace from github.com/codespaces, or run **Codespaces: Stop Current Codespace** from the command palette. When you have pushed everything you care about, delete it, because storage bills while it merely exists.

Optional · VS Code Desktop instead of the browser

This fixes the keystroke conflicts and the large-paste problem, and it needs a local install, which is probably why you are reading this document. Try it only if VS Code is already on your machine or is in your company's self-service catalogue.

1. Install the **GitHub Codespaces** extension, publisher GitHub, id `GitHub.codespaces`. VS Code 1.51 or later.
2. Open the **Remote Explorer** in the activity bar, pick **GitHub Codespaces**, then **Sign in to GitHub** and authorise.
3. Click the plug icon beside your codespace to connect. The container is the same one; only the editor changes.
4. To make it the default: **Settings › Codespaces › Editor preference**.

One quirk worth knowing: with Desktop connected, a private forwarded port is also reachable at plain `127.0.0.1:5173` on your laptop, so `localhost` starts working again.

The morning checklist

Ten minutes if the codespace already exists. Twenty if you are creating it now.

1. Open `github.com/codespaces` and start the codespace. Wait for the terminal.
2. `node --version`, `python3 --version`, `uv --version`, `git --version`. All four print a version.
3. `claude --version` and `codex --version`. If either is missing you rebuilt at some point: redo Step 2.
4. Start the agent in your own folder: `cd /workspaces/my-habits && claude`. Accept the trust prompt.
5. Confirm you are logged in. If not, Step 3, and expect the paste-the-code flow.
6. Run `/terminal-setup` once so Shift+Enter gives you a newline. Ctrl+J works regardless.
7. In the net: `cd /workspaces/tca-habits && ./scripts/dev.sh`. Open port 5173 from the PORTS tab and confirm the app renders.
8. Ctrl-C it. You want the net proven, not running.
9. Create `prompt.txt` in your app folder now. You will paste the day's long prompts into it rather than into the terminal.
10. One sentence to bring: a habit you actually want to build.

When it goes wrong

Symptom	Cause and fix
"Create codespace" is missing or fails	The repository is empty, which GitHub cannot create a codespace for. Or you are out of allowance: check <code>github.com/settings/billing</code> .
Browser shows a login code and nothing happens	Expected. Paste the code at the terminal prompt. If the prompt is unresponsive, <code>claude auth login</code> .
Codex login opens a browser and hangs	The callback to <code>localhost:1455</code> cannot reach the container. Use <code>codex login --device-auth</code> .
Codex warns about the sandbox at startup	<code>bwrap</code> cannot nest here. Run <code>codex --sandbox danger-full-access</code> .
Blocked request. This host is not allowed	Vite's host check. Add <code>allowedHosts: ['.app.github.dev']</code> to the server config.
Forwarded URL returns 502 or hangs	Nothing is listening. Check the process is still running in the terminal and that you opened 5173, not 8000.
claude and uv have vanished	You rebuilt the container. Redo Step 2 and Step 3, and re-run <code>/terminal-setup</code> . Your code in <code>/workspaces</code> is untouched.
<code>gh repo create</code> is refused	The codespace token is scoped to the repository the codespace came from. Run <code>gh auth login</code> , or create the repo in the web UI and add the remote by hand.

Symptom	Cause and fix
Everything stopped mid-lab	Idle timeout after 30 minutes of silence. Click the codespace name to restart. Uncommitted work is intact.
The terminal eats a long prompt	Documented behaviour for the VS Code terminal. Put the prompt in a file and have the agent read it.

Worth bookmarking

- docs.github.com/en/codespaces · ports, timeouts, retention, persistence
- github.com/codespaces · start, stop, keep and delete your codespaces
- code.claude.com/docs/en/authentication · the paste-the-code login flow, and setup-token
- code.claude.com/docs/en/devcontainer · what persists, and the Codespaces secret recipe
- developers.openai.com/codex/auth · sign-in options, and where Codex keeps your credentials
- developers.openai.com/codex/agent-approvals-security · why to use danger-full-access inside a container

These pages move, and both CLIs ship changes most weeks. This document was checked against the vendor documentation in August 2026. Trust the docs over any handout, including this one.